Laboratorio logs del sistema en Linux con journalctl

La llegada de **systemd** a la mayoría de distribuciones de GNU/Linux, como sistema de inicio reemplazando a init, ha supuesto una nueva manera de ver de los mensajes del sistema, como pueden ser los del kernel y los diferentes servicios o procesos.

Journalctl es ahora la herramienta utilizada para acceder a los registros del sistema y en este laboratorio vamos a ver cómo trabajar con la utilidad journalctl.

Por defecto todos los mensajes generados por systemd se guardan como registros journal en **/run/log/journal/**, este directorio por defecto se vacía en cada reinicio del servidor, es decir no es persistente.

Uso básico de journalctl

La forma básica de acceder a los **registros del sistema** es:

```
journalctl
```

que viene a equivaler al viejo *"cat /var/log/messages"* y nos permite acceder a **todos los logs sin filtrar** (en cualquier momento podemos salir de esa pantalla utilizando la tecla "q")

Como pasa con el resto de comandos podemos **redirigir la salida hacia un documento de texto**

```
journalctl > mensajes.log
```

Si queremos seguir los **mensajes en tiempo real** añadimos el parámetro –f

```
journalctl -f
```

Filtrar la salida de logs por boots

Si habéis ejecutado alguno de los comandos anteriores os habréis dado cuenta que el número de mensajes es inmenso, así que vamos a filtrarlos por los diferentes **procesos de arranque,** que haya tenido nuestro ordenador.

Cada arranque (boot) de sistema tiene un nº (0 para el actual) e ID asignado

Para ver la **lista de todos los boots:**

```
journalctl --list-boots
```

```
Archivo  Editar  Ver  Buscar  Terminal  Ayuda
-25 7d329598760e4ef2b552e487757356b6 jue 2015-03-19 10:01:54 CET—jue 2015-03-19 12:39:35 CET
-24 7ae23d043d6248819ccfb63f5bafdbdd jue 2015-03-19 17:43:55 CET—vie 2015-03-20 01:16:35 CET
-23 071cac06eaaa4340b048a48952fe26a4 vie 2015-03-20 10:35:17 CET—vie 2015-03-20 19:56:40 CET
-22 0089eeafdcc44f449d5309c61af5e2dc vie 2015-03-20 21:25:15 CET—sáb 2015-03-21 01:41:06 CET
-21 4aa717c3ebd441ee93417a5caf6a3aab sáb 2015-03-21 11:40:03 CET—sáb 2015-03-21 14:32:02 CET
-20 5aab3ac811854d389bfc7037e513af0d sáb 2015-03-21 20:05:08 CET—dom 2015-03-22 02:06:03 CET
-19 7c81ffa660b9477cb5220f7024227f0f dom 2015-03-22 11:56:20 CET—lun 2015-03-23 01:39:47 CET
-18 546e427e87084026be37cfc9156cc9e1 lun 2015-03-23 09:32:52 CET—mar 2015-03-24 02:00:05 CET
-17 ac2a3da2a4d2464e909479e252eb5043 mar 2015-03-24 10:38:27 CET—mar 2015-03-24 22:48:06 CET
-16 fffd91d2b06c42c594ba8b6e89023f64 mar 2015-03-24 22:48:31 CET—mié 2015-03-25 02:30:16 CET
-15 25bdd8fa8e8a4d77b134b65a98a89300 mié 2015-03-25 10:45:07 CET—mié 2015-03-25 20:52:11 CET
-14 dc0a79eb5c7b40ba94c0c0f0b431804c mié 2015-03-25 20:52:43 CET—jue 2015-03-26 00:04:46 CET
-13 3c953a7bcf6b4d55b5ee02bd2d77eeae jue 2015-03-26 00:05:11 CET—jue 2015-03-26 01:02:35 CET
-12 ee7d71261bca421581501dead14260e6 jue 2015-03-26 08:28:16 CET—jue 2015-03-26 17:25:43 CET
-11 bb8ee6d7b3bc42ee993fd1f0c7db5e58 jue 2015-03-26 17:26:15 CET—jue 2015-03-26 17:28:20 CET
-10 0ec7438ff6aa425eb7658fc0f3ce4cf5 jue 2015-03-26 17:28:46 CET—jue 2015-03-26 17:40:28 CET
 -9 d7db29a5728a4d18bd854dfb68f437d8 jue 2015-03-26 17:40:54 CET—jue 2015-03-26 17:50:09 CET
 -8 48979d9675e947aabb062a46962f8d9a jue 2015-03-26 17:50:35 CET—jue 2015-03-26 18:10:41 CET
 -7 0e18f49fd3824a75812c617bfeebdbaf jue 2015-03-26 18:11:13 CET—jue 2015-03-26 18:13:52 CET
 -6 c77f5b7eecdd44c0b960518432a62da6 jue 2015-03-26 18:14:17 CET—jue 2015-03-26 18:29:27 CET
 -5 b65513b128b74a97a0cefdbe516df397 jue 2015-03-26 18:29:58 CET—jue 2015-03-26 20:25:51 CET
 -4 8962c86f57984ff7badd03dfa0af9e88 jue 2015-03-26 20:26:45 CET—vie 2015-03-27 02:48:52 CET
 -3 a94fc59c7fc94857bf2de498017e2fc5 vie 2015-03-27 11:12:32 CET—sáb 2015-03-28 01:44:38 CET
 -2 5d21fb0f5c9c453392f424158aef35d6 sáb 2015-03-28 11:06:06 CET—sáb 2015-03-28 13:25:35 CET
 -1 dfe74d5a9d384a88821ff8d24d64f81f sáb 2015-03-28 18:06:51 CET—dom 2015-03-29 04:13:34 CEST
  0 99c0e8116423417f91d048cc0db35f11 dom 2015-03-29 13:34:17 CEST—dom 2015-03-29 16:53:01 CEST
jose@replicante:~>
```

Para ver los **logs del boot actual, es decir del arranque del servidor, podemos reiniciar el servidor y comprobar los comandos:**

```
journalctl -b
```

Si necesitamos ver alguno de los arranques del sistema operativo **anteriores** tenemos dos opciones:

- Utilizamos simplemente una **cuenta regresiva**. Por ej para ver al anterior:

```
journalctl -b -1
```

y así sucesivamente:

```
journalctl -b -2

journalctl -b -3
```

- O elegimos usar la **ID del boot**, que nos aparecio al listar los procesos de arranque con "**journalctl –list-boots**". Sería algo así:

```
journalctl _BOOT_ID=dfe74d5a9d384a88821ff8d24d64f81f
```

Ver los mensajes del kernel

Le añadimos el **parámetro -k:**

```
journalctl -k
```

Como suele pasar en la linea de comandos podemos combinar varios parámetros para afinar la búsqueda.

Aquí vemos los mensajes referidos al **kernel durante el boot actual**:

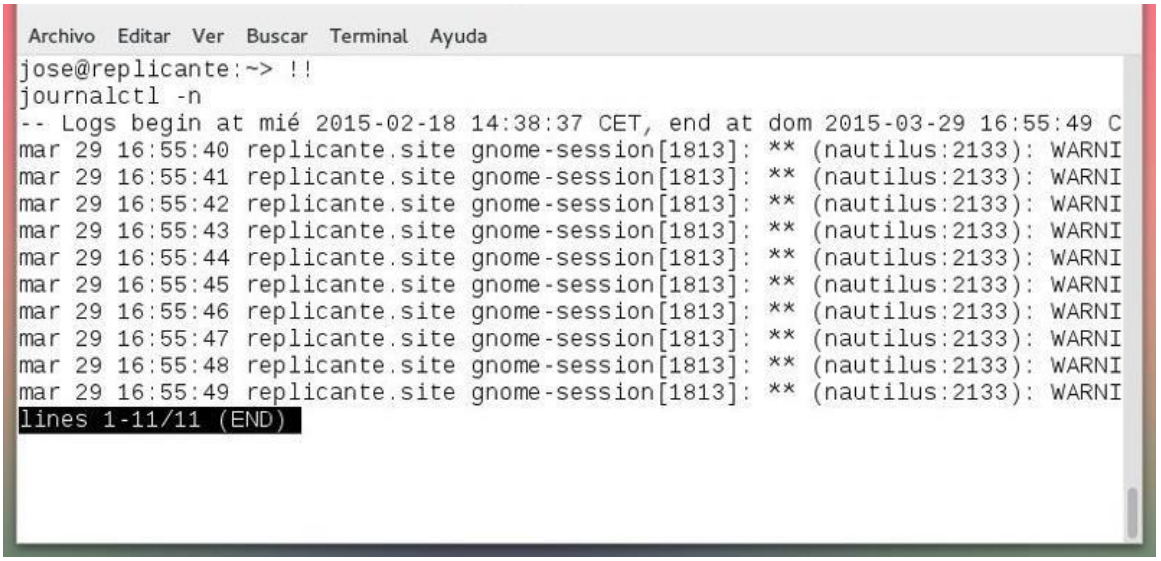
```
journalctl -k -b
```

Filtrar por número de entradas en el registro de logs

La opción predeterminada es ejecutar

```
journalctl -n
```

la cual nos proporciona los **últimos 10 mensajes**



Pero lo podemos cambiar, simplemente poniendo un número:

```
journalctl -n 50
```

Filtras los logs por ejecutables o programas

En este caso también tenemos varias formas de hacerlo, bien directamente con el **ejecutable**:

```
journalctl _COMM=NetworkManager
```

o especificando la **ruta**

```
journalctl /usr/sbin/NetworkManager
```

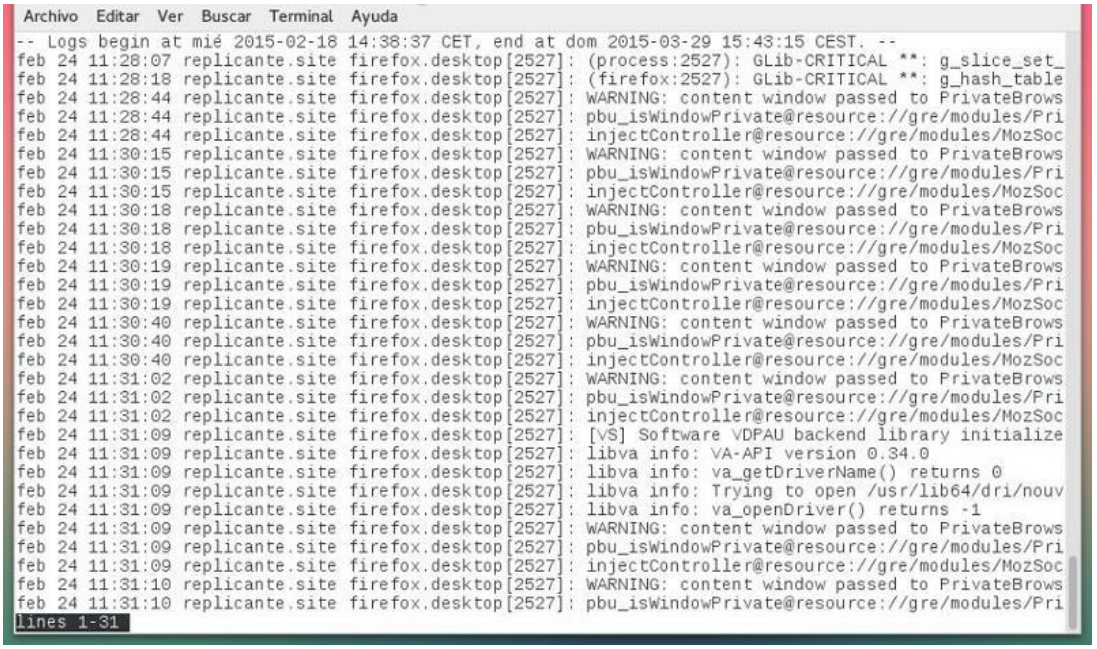
y como siempre tenemos la opción de **filtrar con grep:**

```
journalctl | grep NetworkManager
```

Mostrar la salida por PID

Filtramos mediante el número **identificador del proceso** (algo que podemos consultar con top o htop), en esta ocasión como veis en el ejemplo 2527 corresponde a Firefox:

```
journalctl _PID=2527
```



```
Archivo  Editar  Ver  Buscar  Terminal  Ayuda
-- Logs begin at mié 2015-02-18 14:38:37 CET, end at dom 2015-03-29 15:43:15 CEST. --
feb 24 11:28:07 replicante.site firefox.desktop[2527]: (process:2527): GLib-CRITICAL **: g_slice_set_
feb 24 11:28:18 replicante.site firefox.desktop[2527]: (firefox:2527): GLib-CRITICAL **: g_hash_table
feb 24 11:28:44 replicante.site firefox.desktop[2527]: WARNING: content window passed to PrivateBrows
feb 24 11:28:44 replicante.site firefox.desktop[2527]: pbu_isWindowPrivate@resource://gre/modules/Pri
feb 24 11:28:44 replicante.site firefox.desktop[2527]: injectController@resource://gre/modules/MozSoc
feb 24 11:30:15 replicante.site firefox.desktop[2527]: WARNING: content window passed to PrivateBrows
feb 24 11:30:15 replicante.site firefox.desktop[2527]: pbu_isWindowPrivate@resource://gre/modules/Pri
feb 24 11:30:15 replicante.site firefox.desktop[2527]: injectController@resource://gre/modules/MozSoc
feb 24 11:30:18 replicante.site firefox.desktop[2527]: WARNING: content window passed to PrivateBrows
feb 24 11:30:18 replicante.site firefox.desktop[2527]: pbu_isWindowPrivate@resource://gre/modules/Pri
feb 24 11:30:18 replicante.site firefox.desktop[2527]: injectController@resource://gre/modules/MozSoc
feb 24 11:30:19 replicante.site firefox.desktop[2527]: WARNING: content window passed to PrivateBrows
feb 24 11:30:19 replicante.site firefox.desktop[2527]: pbu_isWindowPrivate@resource://gre/modules/Pri
feb 24 11:30:19 replicante.site firefox.desktop[2527]: injectController@resource://gre/modules/MozSoc
feb 24 11:30:40 replicante.site firefox.desktop[2527]: WARNING: content window passed to PrivateBrows
feb 24 11:30:40 replicante.site firefox.desktop[2527]: pbu_isWindowPrivate@resource://gre/modules/Pri
feb 24 11:30:40 replicante.site firefox.desktop[2527]: injectController@resource://gre/modules/MozSoc
feb 24 11:31:02 replicante.site firefox.desktop[2527]: WARNING: content window passed to PrivateBrows
feb 24 11:31:02 replicante.site firefox.desktop[2527]: pbu_isWindowPrivate@resource://gre/modules/Pri
feb 24 11:31:02 replicante.site firefox.desktop[2527]: injectController@resource://gre/modules/MozSoc
feb 24 11:31:09 replicante.site firefox.desktop[2527]: [VS] Software VDPAU backend library initialize
feb 24 11:31:09 replicante.site firefox.desktop[2527]: libva info: VA-API version 0.34.0
feb 24 11:31:09 replicante.site firefox.desktop[2527]: libva info: va_getDriverName() returns 0
feb 24 11:31:09 replicante.site firefox.desktop[2527]: libva info: Trying to open /usr/lib64/dri/nouv
feb 24 11:31:09 replicante.site firefox.desktop[2527]: libva info: va_openDriver() returns -1
feb 24 11:31:09 replicante.site firefox.desktop[2527]: WARNING: content window passed to PrivateBrows
feb 24 11:31:09 replicante.site firefox.desktop[2527]: pbu_isWindowPrivate@resource://gre/modules/Pri
feb 24 11:31:09 replicante.site firefox.desktop[2527]: injectController@resource://gre/modules/MozSoc
feb 24 11:31:10 replicante.site firefox.desktop[2527]: WARNING: content window passed to PrivateBrows
feb 24 11:31:10 replicante.site firefox.desktop[2527]: pbu_isWindowPrivate@resource://gre/modules/Pri
lines 1-31
```

Especificar la salida por usuarios

La **id de los usuario**s la podéis encontrar ejecutando (obviamente sustituís la parte del nombre del usuario)

```
id nombre_del_usuario
```

y después seleccionáis la que corresponda (1000, 1001, 1002…)

```
journalctl _UID=1001
```

Filtrar la salida por servicios de systemd

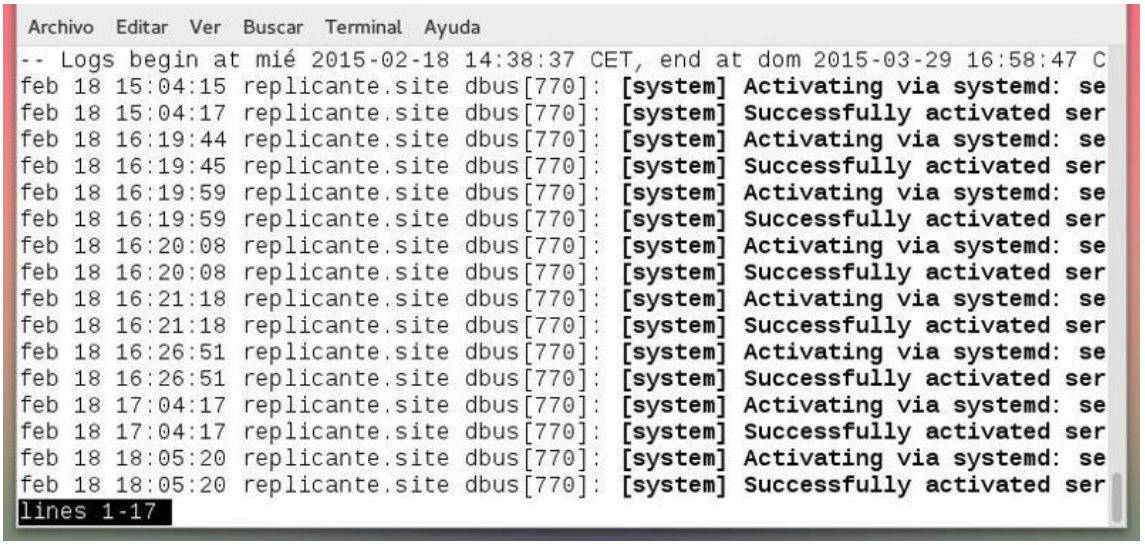
Podemos **ver los servicios** que dependen de systemd, ejecutando:

```
systemctl list-units -t service --all
```

y si nos interesa **uno en particular**, estudiamos sus mensajes añadiendo el **parámetro**

-u y el nombre del service, como en este ejemplo

```
journalctl -u dbus.service
```



Podemos hacer una búsqueda de **varios servicios** añadiendo el caracter "+" (equivale a un "or") y ejecutando:

```
journalctl _SYSTEMD_UNIT=httpd.service + _SYSTEMD_UNIT=dbus.service
```

Filtrar por fechas

Se utilizan los parámetros **"–since**" y **"–until**", así como expresiones tipo "yesterday" "ago" o "today". El formato de tiempo es habitualmente **YYYY-MM-DD HH:MM:SS**.

Algunos ejemplos:

 # Filtro por horas, fechas...
 # Utilizando formato YYYY-MM-DD HH:MM:SS
 # o comodines como yesterday, today, tomorrow o now, (ayer, hoy, mañana, ahora)

```
journalctl --since "2014-03-15 18:20:00"
journalctl --since "2014-03-15" --until "2014-08-20 05:00"
journalctl --since=09:00 --until=16:30
journalctl --since=yesterday
```

- Ver los logs desde el día de **ayer** hasta hoy a las 02:00 horas.

```
journalctl --since 'yesterday' --until '02:00'
```

- un **intervalo de fechas** específico:

```
journalctl --since='2015-02-29 00:01' --until='2015-03-29 00:01'
```

- Todos los logs especificando la fecha en el formato que os comentaba anteriormente, y **filtrando por el programa** Firefox

```
journalctl _COMM=firefox --since='2015-02-29 00:01' --until='2015-03-29 00:01'
```

- Lo mismo que el caso anterior pero en este caso **filtrando por el servicio** de sshd.service

```
journalctl -u sshd.service --since='2015-02-29 00:01' --until='2015-03-29 00:01'

journalctl -u httpd --since=00:00 --until=9:30
```

- O simplemente ver lo que ha pasado en la **última media hora**:

```
journalctl --since '30 min ago'
```

- Par ver los logs del servicio vsftpd y firewald desde hace dos días hasta ahora:

```
journalctl -u vsftpd.service -u firewalld.service --
since "2 days ago"
```

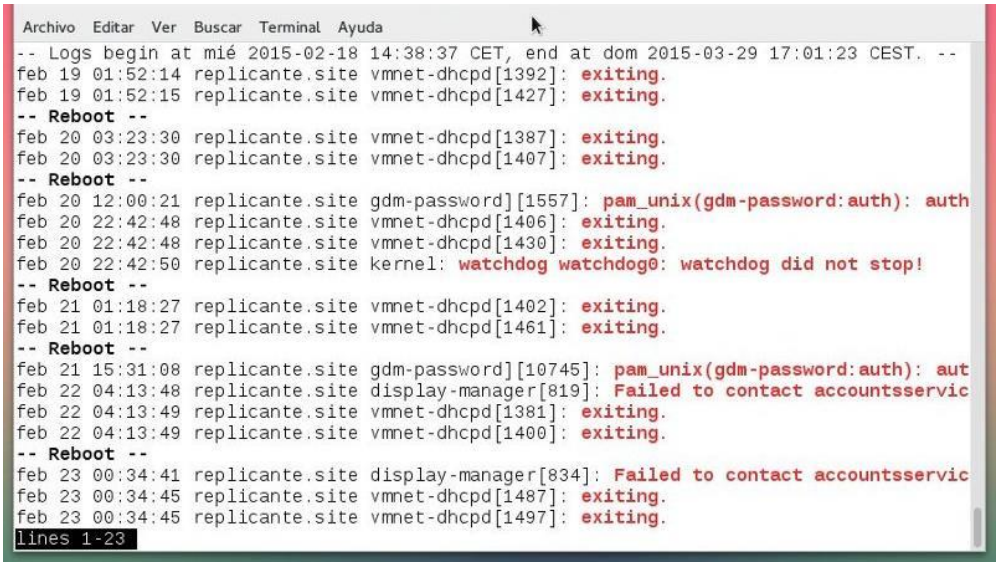
Filtrar por la prioridad del mensaje

Los mensajes se clasifican en función de su prioridad en **7 niveles diferentes**: “emerg” (0),”alert” (1), “crit” (2), “err” (3), “warning” (4), “notice” (5),”info” (6), y “debug” (7)

Para filtrarlos utilizamos el parámetro -p seguido del número correspondiente. En este ejemplo vemos los críticos:

```
journalctl -p 2
```

```
journalctl -p err
```



Mensajes de los discos duros

Puede ser interesante para descubrir problemas de funcionamiento en nuestras unidades de **discos duros**.

```
journalctl /dev/sda
```

Ver el espacio que están ocupando los diferentes logs

Ejecutamos el comando:

```
journalctl --disk-usage
```

En el caso de que ese valor fuera excesivo lo podemos limitar en el archivo **/etc/systemd/journald.conf**,

¿Quién puede ver los registros que va recopilando journald?

En un principio y como es tradición en los sistemas Linux, root tiene acceso a todo. Un usuario normal únicamente podrá ver los registros propios, pero nada más a nivel de sistema. Para tener acceso a estos datos tendremos que añadir al usuario al grupo `adm`: Tenga en cuenta que el control de acceso sólo funciona cuando el almacenamiento persistente está habilitado para journal

```
usermod -a -G adm [usuario]
```

Enabling Persistent Storage

By default, Journal stores log files only in memory or a small ring-buffer in the **/run/log/journal**/ directory. This is sufficient to show recent log history with journalctl. This directory is volatile, log data is not saved permanently. With the default configuration, syslog reads the journal logs and stores them in the /var/log/ directory. With persistent logging enabled, journal files are stored in /var/log/journal which means they persist after reboot. Journal can then replace rsyslog for some users (but see the chapter introduction).

Enabled persistent storage has the following advantages

Richer data is recorded for troubleshooting in a longer period of time

For immediate troubleshooting, richer data is available after a reboot

Server console currently reads data from journal, not log files

Persistent storage has also certain disadvantages:

Even with persistent storage the amount of data stored depends on free memory, there is no guarantee to cover a specific time span

More disk space is needed for logs

To enable persistent storage for Journal, create the journal directory manually as shown in the following example. As root type:

#mkdir -p /var/log/journal/

#systemd-tmpfiles --create --prefix /var/log/journal

#chown root:systemd-journal /var/log/journal

#chmod 2775 /var/log/journal

Then, restart journald to apply the change:

#systemctl restart systemd-journald

Configurando Journal

Disponemos del fichero de configuración **/etc/systemd/journald.conf**, con las siguientes opciones:

Storage

Aquí indicamos cómo queremos que se almacene el registro. Los posibles valores son:

- Volatile: En este caso, el registro se almacena en /run/systemd/journal y es volátil (perdemos los registros en cada reinicio del equipo).
- Persistent: Justo lo contrario al valor anterior: Se almacena en disco, en /var/log/journal.
- Auto: Es similar a persistent, con la difirencia que, si no existe /var/log/journal, no se va a crear y se utilizará /run/systemd/journal, es decir, los logs serán volátiles. En el caso de Arch, ésta es la opición predeterminada y, aunque al instalar o reinstalar el paquete systemd, se crea el /var/log/journal, si lo borramos, los logs serán volátiles hasta la siguiente actualización.
- None: No se almacenan los logs. Es útil cuando queremos redirigirlos a consola, un buffer o alguna implementación de syslog.

Compress

Por defecto está habilitado y comprimirá los datos recogidos.

Seal

Está habilitado por defecto y nos va a permitir proteger los archivos los archivos de regsitro de alteraciones inadvertidas. Es decir, nos puede ser útil para poder afirmar que el registro no ha sido modificado, cosa que no podíamos afirmar con el clásico sistema de logs en texto plano. Para hacer esto, debemos crear las claves con jornalctl –setup-keys

SplitMode

Nos va a permitir especifiar cómo queremos que se dividan los resgistros por usuario:

- Uid: Cada usuario tendrá sus propios archivos de registro (por defecto).
- Login: Cada usuario que inicie sesión tendrá su propio archivo de registro. Los logs de los usuarios que no disponen de sesión interactiva, se registrarán en el log del sistema.
- None: Sólo existe un archivo, no se va a reailizar ninguna división.

RateLimitInterval, RateLimitBurst

Nos permiten limitar el número de mensajes que vamos a permitir almacenar por cada servicio, en un período de tiempo. Si se sobrepasan los mensajes por período de tiempo, se pierden y se genera un post con el número de mensajes perdidos. Los valores son de 1000 mensajes en 30 segundos. Se puede desactivar la limitación, estableciendo los valores a 0.

SystemMaxUse, SystemKeepFree, SystemMaxFileSize, RuntimeMaxUse, RuntimeKeepFree, RuntimeMaxFileSize

Estas opciones nos van a permitir limatar el tamaño el tamaño en disco (para el almacenamiento persistente) o en RAM (para el alamacenamiento volátil). Las opciones con prefijo System hacen referencia al disco y, con prefijo Runtime, a la RAM. Nos permiten “rotar” los archivos, para no tener archivos con un crecimiento sin límites.

MaxFileSec

Nos permite establecer una periocidad para la rotación de archivos, además de la vista anteriormente.

MaxRetentionSec

Nos permite establecer el tiempo máximo que vamos a gruardar las entradas antiguas.

SyncIntervalSec

Es el tiempo de espera antes de sincronizar los archivos de registro en el disco. Después de la sincronización, los archivos de registro se colocan en el estado OFFLINE. Esta configuración sólo se aplica a los niveles ERR, WARNING, NOTICE, INFO, DEBUG y está establecida a cinco minutos por defecto.

ForwardToSyslog, ForwardToKMsg, ForwardToConsole, ForwardToWall

Nos permiten redirigir los logs. Por defecto, Syslog y Wall están habilitados. Por supuesto, si no disponemos de ningún daemon syslog corriendo en nuestro sistema, no tendrá ningún efecto. La opción **ForwardToConsole** nos permite dirigir los mensajes a una consola y, **ForwardToWall**, a todos los usuarios que tengan sesión iniciada en el sistema.

MaxLevelStore, MaxLevelSyslog, MaxLevelKMsg, MaxLevelConsole, MaxLevelWall

Estas opciones nos permiten controlar el tipo de mensaje, por nivel, que se van a redirigir.

TTYPath

La ruta donde queremos que se envíen los mensajes, cuando tenemos **ForwardToConsole=yes**. El valor predeterminado es **/dev/console**.

Rotar el log en Journal

Por defecto, (cuando **Storage=** está definido como **auto** en **/etc/systemd/journald.conf**), journal escribe en /var/log/journal/. Si el directorio **/var/log/journal**/ no existe (por ejemplo, si lo ha eliminado usted o algún programa), **systemd no lo crea** de forma automática, sino que escribe los registros en /run/systemd/journal. Esto significa que los registros se perderán al reiniciar

Límite del tamaño de journal

Si journal se ha creado como permanente (no volátil), el límite de su tamaño se establece con un valor predeterminado correspondiente al 10% del tamaño del sistema de archivos.

Por ejemplo, con `/var/log/journal` alojado en una partición raíz de 50 GiB, esto permitiría almacenar hasta 5 GiB de datos en journal. El tamaño máximo del journal permanente puede ser controlado por `SystemMaxUse` en `/etc/systemd/journald.conf`, por lo que, para limitarlo, por ejemplo, a 50 MiB, descomente y modifique la correspondiente línea a:

```
SystemMaxUse=50M
```

Rotamos el log a 50 Megas.

#vi **/etc/systemd/journald.conf**

Descontentamos: `SystemMaxUse=50M`

 # Limpiar registros reteniendo como máximo 2GB

```
journalctl --vacuum-size=2G
```

 # Limpiar registros y retener sólo los 2 últimos años

```
journalctl --vacuum-time=2years
```

Ver toda la línea del texto de salida journalctl

Creamos el siguiente alias en ~/.bashrc:

```
alias errores="journalctl -b -p err|less"
```

Finalmente, podemos utilizar filtros combinados a partir de las distintas opciones que hemos visto anteriormente

```
journalctl -p warning --since "2014-03-15 18:20:00" _UID=1000

journalctl -u sshd --since=00:00 --until=9:30
```